Introduction to Neural Networks and Deep Learning

Pavel Cejnar

February 17, 2026

University of Chemistry and Technology Prague
Faculty of Chemical Engineering
Department of Mathematics, Informatics and Cybernetics

unless otherwise stated

Credits

Deep Learning courses (NPFL114, NPFL138) by Milan Straka

- Institute of Formal and Applied Linguistics, Faculty of Mathematics and Physics, Charles University in Prague
- courses online at https://ufal.mff.cuni.cz/courses/npfl114, https://ufal.mff.cuni.cz/courses/npfl138

The Deep Learning book by Ian Goodfellow, Yoshua Bengio, and Aaron Courville

- is freely available on-line at http://www.deeplearningbook.org
- is also available from MIT Press

The Dive into Deep Learning book by Aston Zhang, Zachary C. Lipton, Mu Li, and Alexander J. Smola

- is freely available on-line at http://www.d2l.ai

Credits

Arxiv.org

- on-line repository for most-recent papers from AI
- https://arxiv.org

Organization

Course Website

https://moodle.vscht.cz/course/view.php?id=55 - Neuronové sítě (Neural Networks)

- Slide templates, assignments, announcements, exam questions.

Practicals

- There will be assignments - to finish/improve/optimize the code from the lecture etc.
- After solving the assignment, you get non-bonus points, and sometimes also bonus points (for leading places from competitions or outstanding solutions).
- There could be a voluntary project for additional non-bonus points upon request.
- **To pass the practicals, you need to get 60 points (be it bonus or non-bonus).** There will be assignments for at least 120 non-bonus points.
- If you get more than 80 points, then **all the points above 80 will be doubled and transferred to the exam**.

AI Assistance when Solving Assignments

- Relying blindly on AI during learning seems to have negative effect on skill acquisition:
  - How AI Impacts Skill Formation
  - Endoscopist deskilling risk after exposure to artificial intelligence in colonoscopy: a multicentre, observational study
- Therefore, you are **not allowed** to directly copy the assignment descriptions to generative AI and you are **not allowed** to directly use or copy-paste source code generated by generative AI.
  - 50% of Italy's Coders Crashed in 48 Hours After the ChatGPT Ban
- However, discussing your manually written code with generative AI is fine.

Lecture

You need to pass a written exam (or have enough points from the assignments).

- All questions will be publicly listed on the course website.
- There are questions for 100 points in every exam, plus the surplus points from the practicals.
- You need 50/60/70/80/90 points to pass with grade E/D/C/B/A; 70 points for PhD students.

Machine Learning

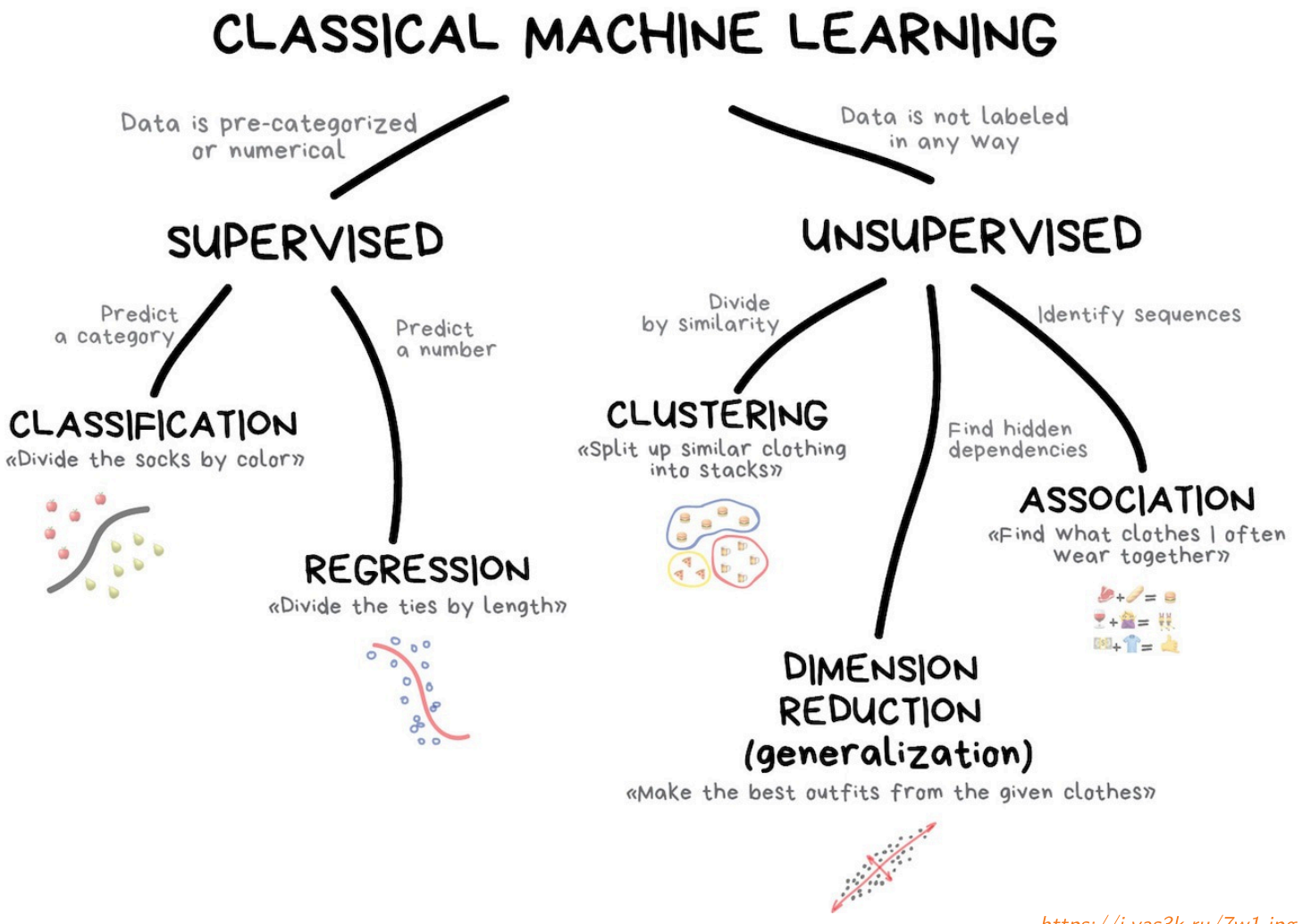
Machine Learning

A possible definition of learning from Mitchell (1997):

> A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.

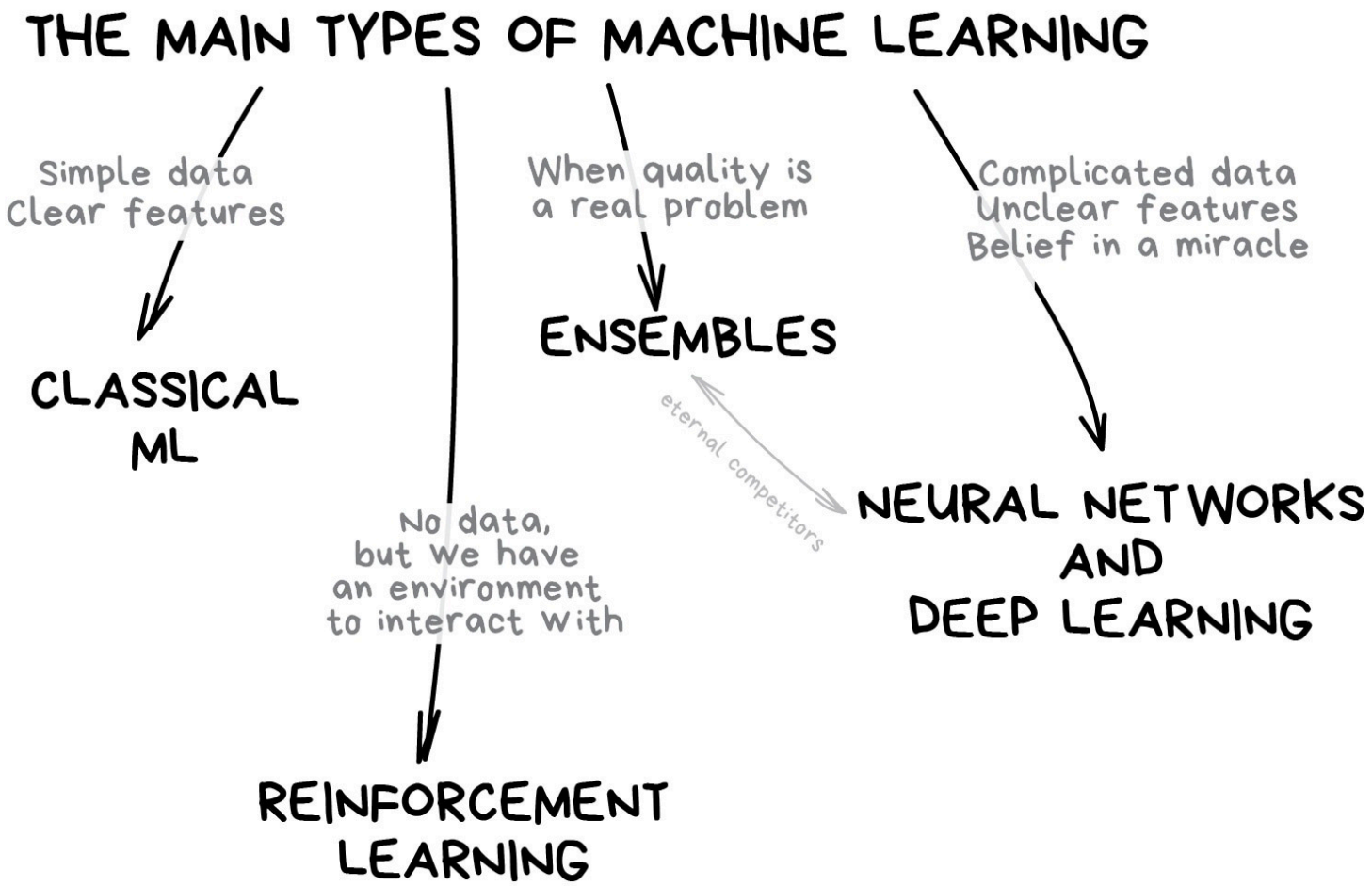
- Task T
  - *classification*: assigning one of k categories to a given input
  - *regression*: producing a number $x \in \mathbb{R}$ for a given input
  - *structured prediction*, *denoising*, *density estimation*, ...
- Measure P
  - *accuracy*, *error rate*, *F-score*, ...
- Experience E
  - *supervised*: usually a dataset with desired outcomes (*labels* or *targets*)
  - *unsupervised*: usually data without any annotation (raw text, raw images, ...)
  - *reinforcement learning*, *semi-supervised learning* (*self-supervised learning*), ...

Classical Machine Learning



https://i.vas3k.ru/7w1.jpg

Main Types of Machine Learning



https://noeliagorod.com/2019/05/21/machine-learning-for-everyone-in-simple-words-with-real-world-examples-yes-again/

What are Neural Networks

Neural networks are just a model for describing computation of outputs from given inputs.

The model:

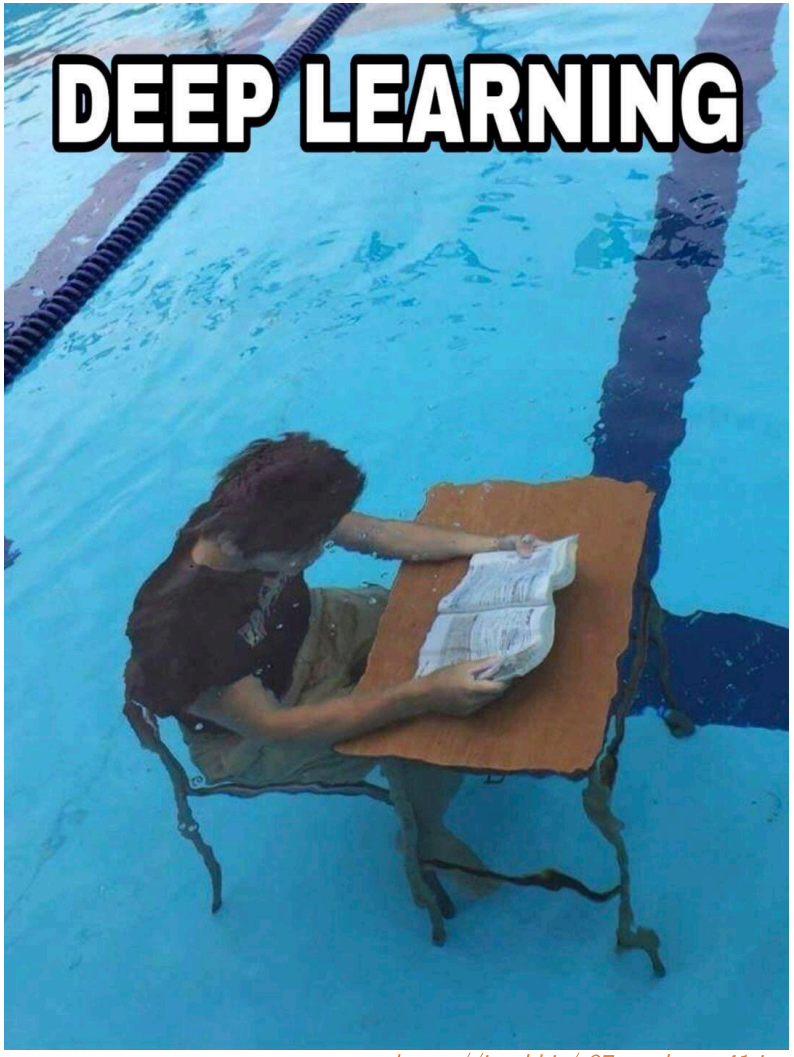
- is strong enough to approximate any reasonable function,
- is reasonably compact,
- allows heavy parallelization during execution (GPUs, TPUs, ...).

Nearly all the time, neural networks generate a *probability distribution* on output:

- distributions allow small changes during training,
- during prediction, we usually take the most probable outcome (class/label/...).

When there is enough data, neural networks are currently the best performing machine learning model, especially when the data are high-dimensional (images, videos, speech, texts, ...).

What is Deep Learning



https://i.redd.it/t87gswsbmnq41.jpg

Deep Learning Highlights

Deep Learning Highlights

- Image recognition, Object detection
- Image segmentation, Human pose estimation
- Image labeling
- Visual question answering
- Lip reading
- Speech recognition and generation
- Machine translation
- Machine translation without parallel data
- Arcade games, Chess, Go, and Shogi (*combined with reinforcement learning*)
- Multiplayer Capture the flag (*combined with reinforcement learning*)
- Large language models (*combined with reinforcement learning*)

Well-known Datasets

Name	Description	Instances
MNIST	Images (28x28, grayscale) of handwritten digits.	60k
CIFAR-10	Images (32x32, color) of 10 classes of objects.	50k
CIFAR-100	Images (32x32, color) of 100 classes of objects (with 20 defined superclasses).	50k
ImageNet	Labeled object image database (labeled objects, some with bounding boxes).	14.2M
ImageNet-ILSVRC	Subset of ImageNet for Large Scale Visual Recognition Challenge, annotated with 1000 object classes and their bounding boxes.	1.2M
COCO	*Common Objects in Context*: Complex everyday scenes with descriptions (5) and highlighting of objects (91 types).	2.5M

ImageNet-ILSVRC

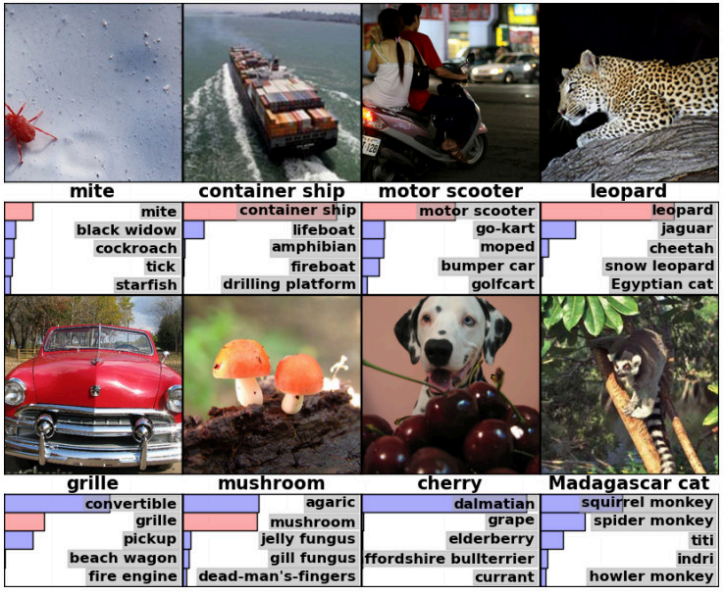
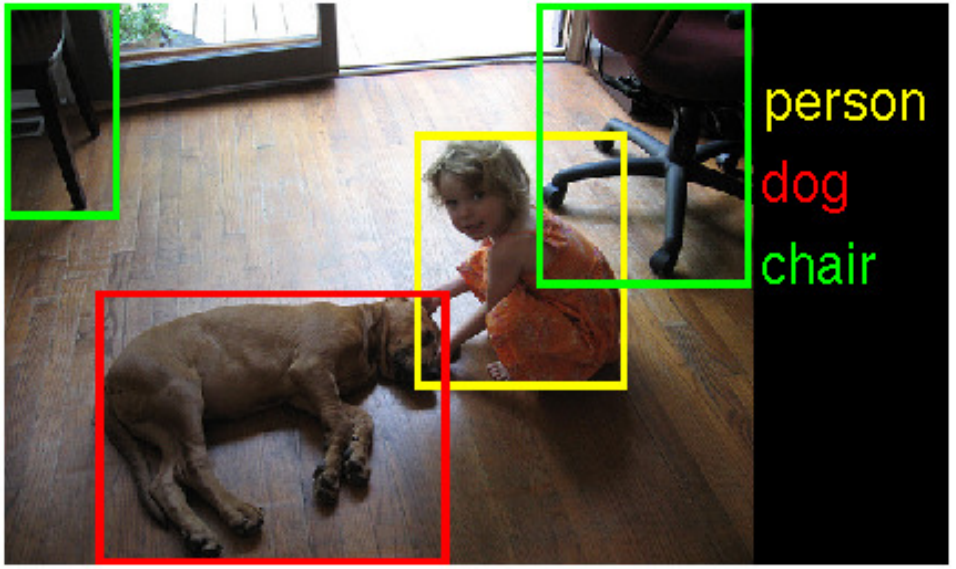


Figure 4 of "ImageNet Classification with Deep Convolutional Neural Networks" by Alex Krizhevsky et al.



https://image-net.org/challenges/LSVRC/2014/

Well-known Datasets

COCO

https://cocodataset.org/#detection-2020

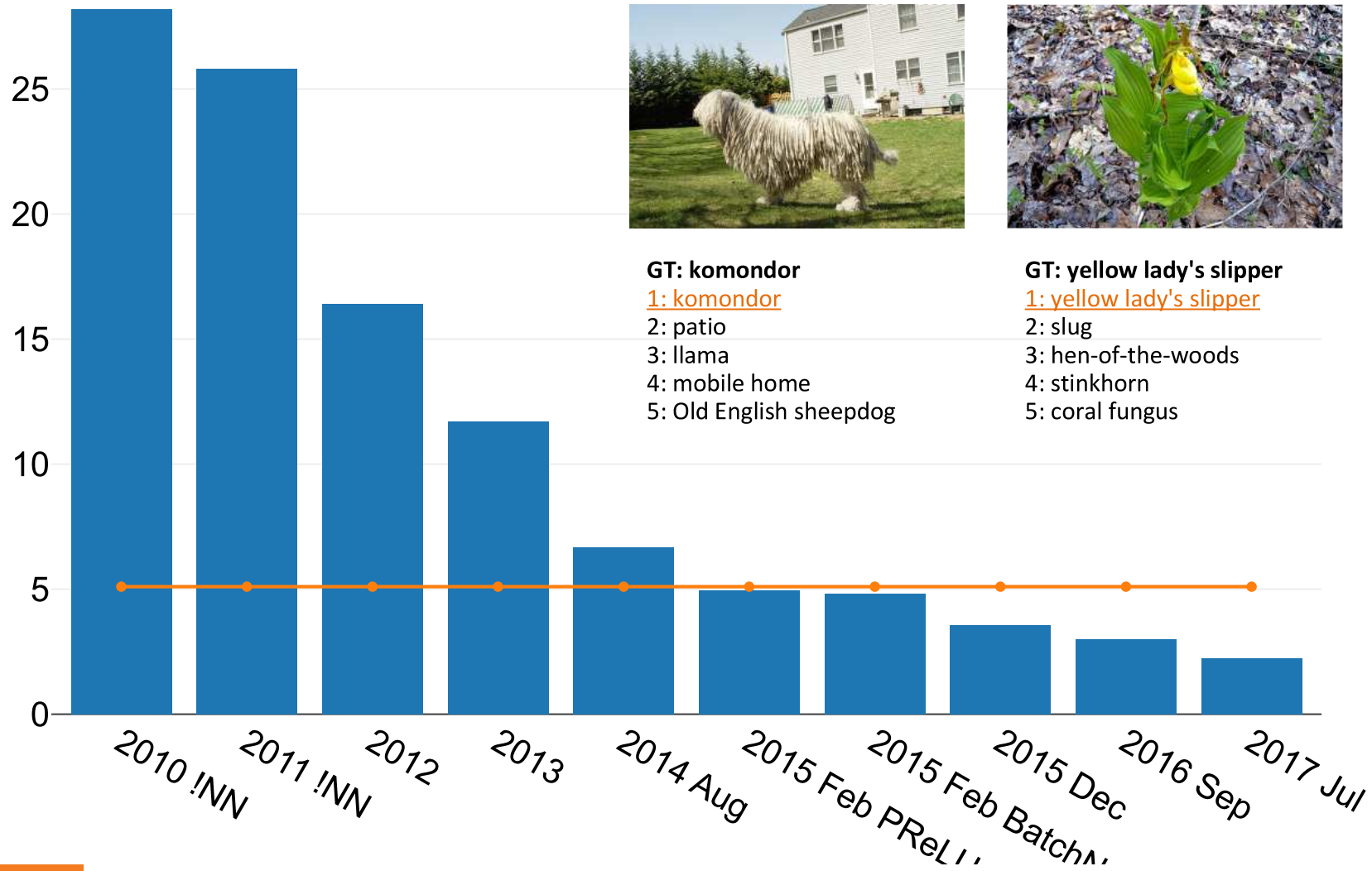
Well-known Datasets

Name	Description	Instances
TIMIT	Recordings of 630 speakers of 8 dialects of American English.	6.3k sents
CommonVoice	2,681 hours of verified English speech from 94k speakers 78 hours of verified Czech speech from 1k speakers	22k hours, 133 langs
PTB	*Penn Treebank*: 2500 stories from Wall Street Journal, with POS tags and parsed into trees.	1M words
PDT-C	*Prague Dependency Treebank – Consolidated*: Czech data annotated on morphological, analytical, tectogrammatical layers.	3.8M words
UD	*Universal Dependencies*: Treebanks of 168 languages with consistent annotation of lemmas, POS tags, morphology, syntax.	296 treebanks
WMT	Aligned parallel sentences for machine translation.	gigawords

Massive Datasets

Name	Description	Instances
Open Images	Images annotated with image-level labels, object bounding boxes, object segmentation masks, visual relationships, and localized narratives.	9M
LAION-400M	A collection of (image, English text) pairs.	400M
LAION-5B	A collection of (image, text) pairs; 2.3B English descriptions, 2.2B from 100+ other languages, 1B unknown (names, ...).	5.85B
FineWeb	A collection of deduplicated and cleaned English web data.	15T tokens, 47.5TB
FineWeb-2	Deduplicated and cleaned web data in 1000+ languages.	3T tokens, 8TB

ILSVRC Image Recognition Top-5 Error Rates



ILSVRC Image Recognition Error Rates

In summer 2017, a paper came out describing automatic generation of neural architectures using reinforcement learning.

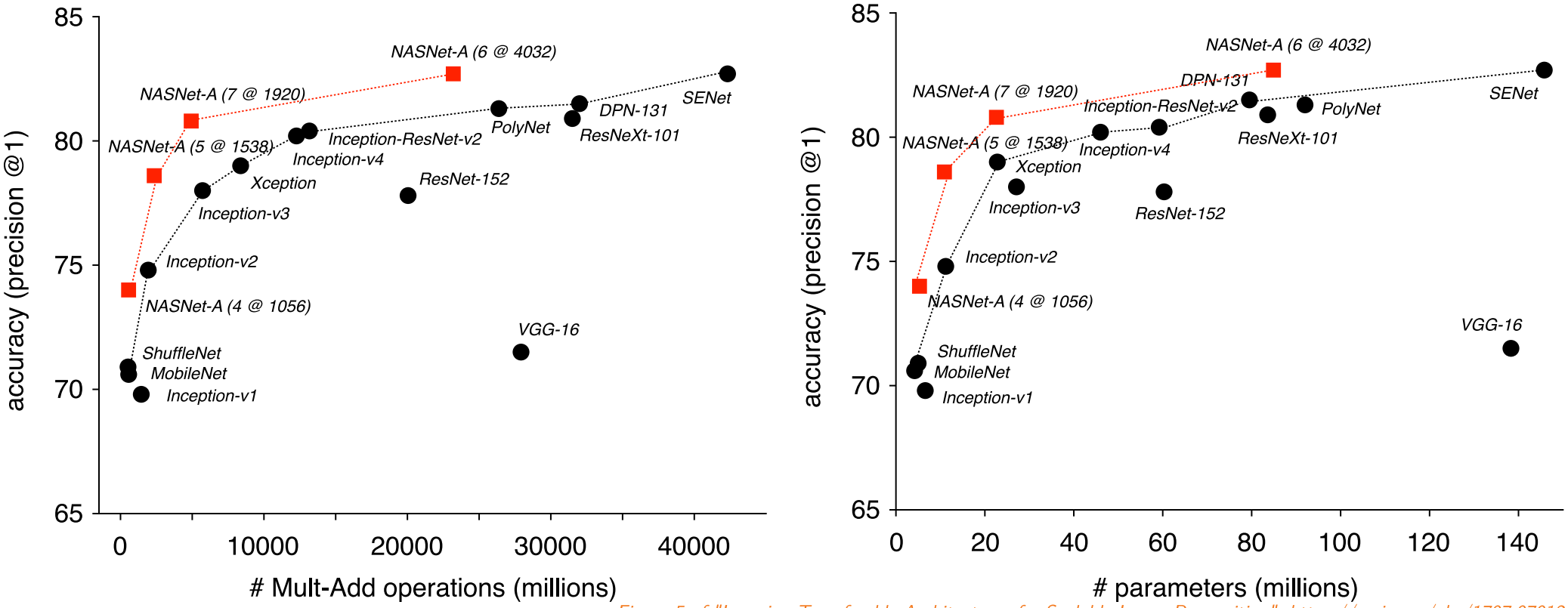


Figure 5 of "Learning Transferable Architectures for Scalable Image Recognition", https://arxiv.org/abs/1707.07012

Machine Translation Improvements

To illustrate deep neural networks improvements in other domains, consider the English→Czech results of the international Workshop on Machine Translation. Both the automatic BLEU metric and manual evaluation are presented.

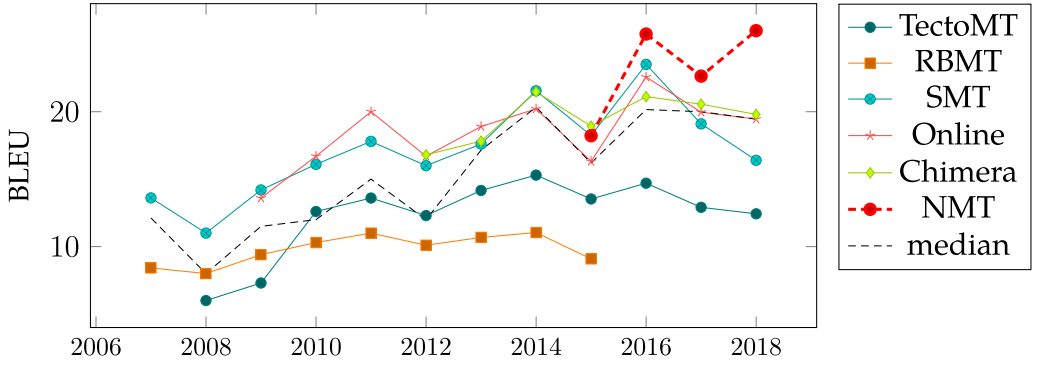


Figure 6.1: WMT English→Czech BLEU evaluation.

Figure 6.1 of "Machine Translation Using Syntactic Analysis", https://dspace.cuni.cz/handle/20.500.11956/104305

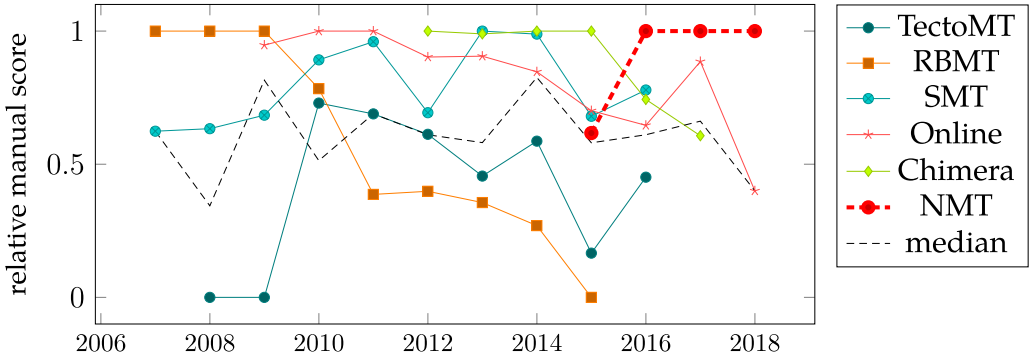


Figure 6.2: WMT English→Czech manual evaluation (higher=better).

Figure 6.2 of "Machine Translation Using Syntactic Analysis", https://dspace.cuni.cz/handle/20.500.11956/104305

- TectoMT parses the input, transfers to the other language, generates the sentence;
- RBMT is the PC-Translator software;
- SMT is statistical machine translation using the Moses system;
- Online is an online translation system (Google in 2009, `Online-B` since 2010);
- **NMT** is the neural machine translation using deep neural networks.

How Good is Current Deep Learning

- DL has seen amazing progress in the last fifteen years.
- Is it enough to get a bigger brain (datasets, models, computer power)?
- Problems compared to Human learning:
  - Sample efficiency
  - Human-provided labels
  - Robustness to data distribution change
  - Stupid errors

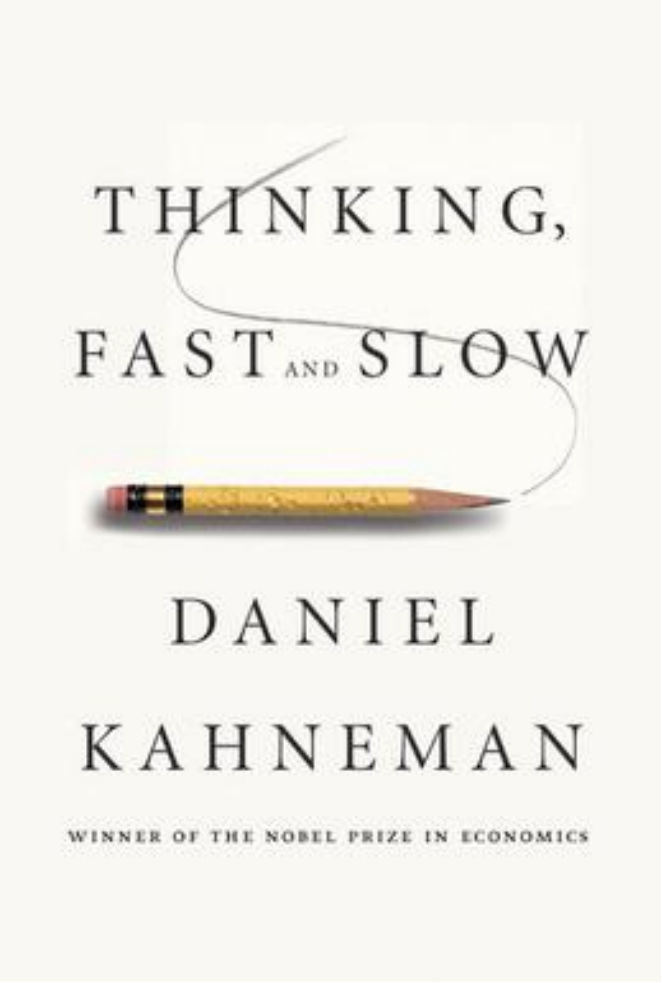
https://intl.startrek.com/sites/default/files/styles/content_full/public/images/2019-07/c8ffe9a587b126f152ed3d89a146b445.jpg

How Good is Current Deep Learning

- Thinking fast and slow
  - System 1
    - intuitive
    - fast
    - automatic
    - frequent
    - unconscious

    Current DL
  - System 2
    - logical
    - slow
    - effortful
    - infrequent
    - conscious

    Future DL



https://en.wikipedia.org/wiki/File:Thinking,_Fast_and_Slow.jpg

How Good is Current Deep Learning

Ilya Sutskever
@ilyasut

it may be that today's large neural networks are slightly conscious

Přeložit Tweet

12:27 dop. · 10. 2. 2022 · Twitter Web App

https://twitter.com/ilyasut/status/1491554478243258368

Yann LeCun
@ylecun

Odpověď uživateli @ilyasut

Nope.
Not even for true for small values of "slightly conscious" and large values of "large neural nets".
I think you would need a particular kind of macro-architecture that none of the current networks possess.

Přeložit Tweet

10:02 odp. · 12. 2. 2022 · Twitter for Android

https://twitter.com/ylecun/status/1492604977260412928

Murray Shanahan
@mpshanahan

Odpověď uživateli @ilyasut

… in the same sense that it may be that a large field of wheat is slightly pasta

Přeložit Tweet

11:08 dop. · 10. 2. 2022 · Twitter Web App

https://twitter.com/mpshanahan/status/1491715721289678848

Curse of Dimensionality

Figure 5.9 of "Deep Learning" book, https://www.deeplearningbook.org

Machine and Representation Learning

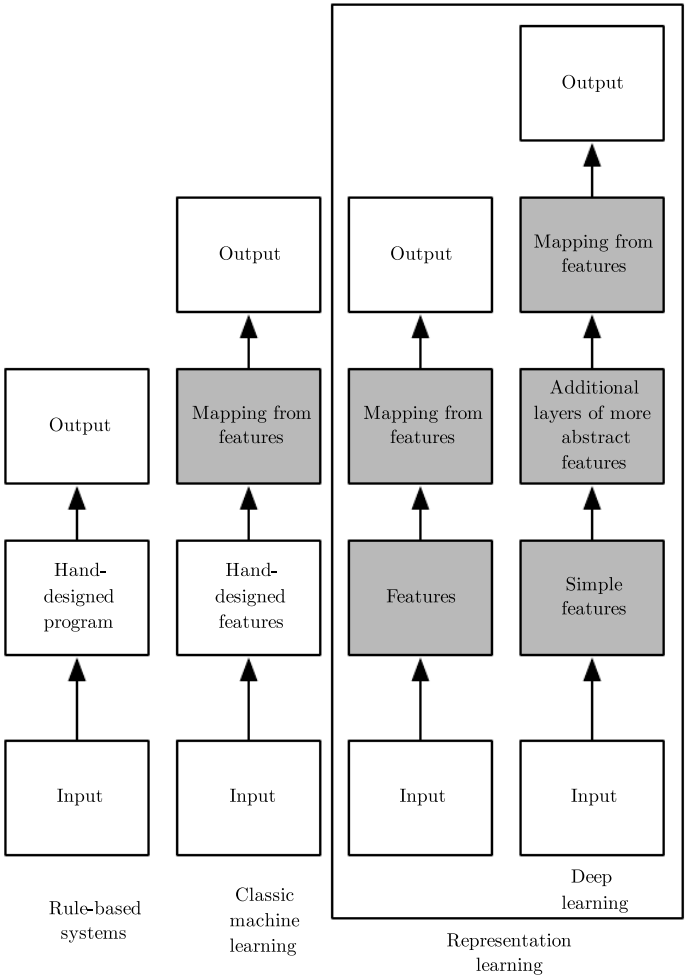


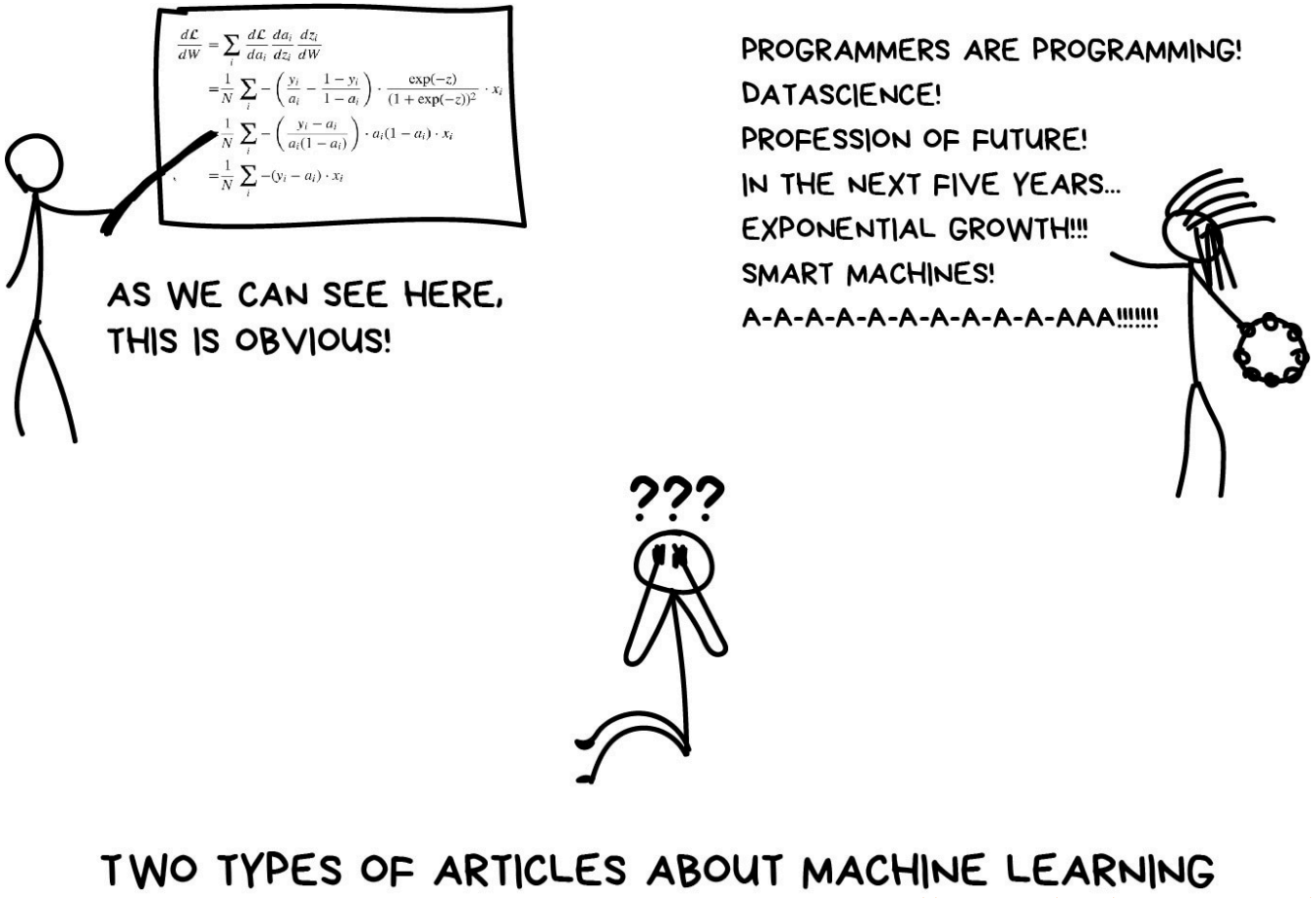
Figure 1.5 of "Deep Learning" book, https://www.deeplearningbook.org

Notation

Notation

- a, $\boldsymbol{a}$, $\boldsymbol{A}$, A: scalar (integer or real), vector, matrix, tensor
  - $c \cdot \boldsymbol{A}$ denotes scalar multiplication, $\boldsymbol{x} \odot \boldsymbol{y}$ denotes element-wise multiplication, and $\boldsymbol{AB}$ denotes matrix multiplication
  - all vectors are always **column** vectors
  - transposition changes a column vector into a row vector, so $\boldsymbol{a}^T$ is a row vector
  - we denote the **dot (scalar) product** of the vectors $\boldsymbol{a}$ and $\boldsymbol{b}$ using $\boldsymbol{a}^T\boldsymbol{b}$
    - we understand it as matrix multiplication
  - the $\|\boldsymbol{a}\|_2$ or just $\|\boldsymbol{a}\|$ is the Euclidean (or L^2) norm
    - $\|\boldsymbol{a}\|_2 = \sqrt{\sum_i a_i^2}$
- a, $\mathbf{a}$, $\mathbf{A}$: scalar, vector, matrix random variable
- $\frac{\partial f}{\partial x}$: partial derivative of f with respect to x
- $\nabla_{\boldsymbol{x}} f(\boldsymbol{x})$: gradient of f with respect to $\boldsymbol{x}$, i.e., $\left(\frac{\partial f(\boldsymbol{x})}{\partial x_1}, \frac{\partial f(\boldsymbol{x})}{\partial x_2}, \ldots, \frac{\partial f(\boldsymbol{x})}{\partial x_n}\right)$

Statistics



https://vas3k.com/blog/machine_learning/

Random Variables

Random Variables

A random variable x is a result of a random process, and it can be either discrete or continuous.

Probability Distribution

A probability distribution describes how likely are the individual values that a random variable can take.

The notation $\mathrm{x} \sim P$ stands for a random variable x having a distribution P.

For discrete variables, the probability that x takes a value x is denoted as $P(x)$ or explicitly as $P(\mathrm{x} = x)$. All probabilities are nonnegative, and the sum of the probabilities of all possible values of x is $\sum_x P(\mathrm{x} = x) = 1$.

For continuous variables, the probability that the value of x lies in the interval $[a, b]$ is given by $\int_a^b p(x)\,\mathrm{d}x$, where $p(x)$ is the *probability density function*, which is always nonnegative and integrates to 1 over the range of all values of x.

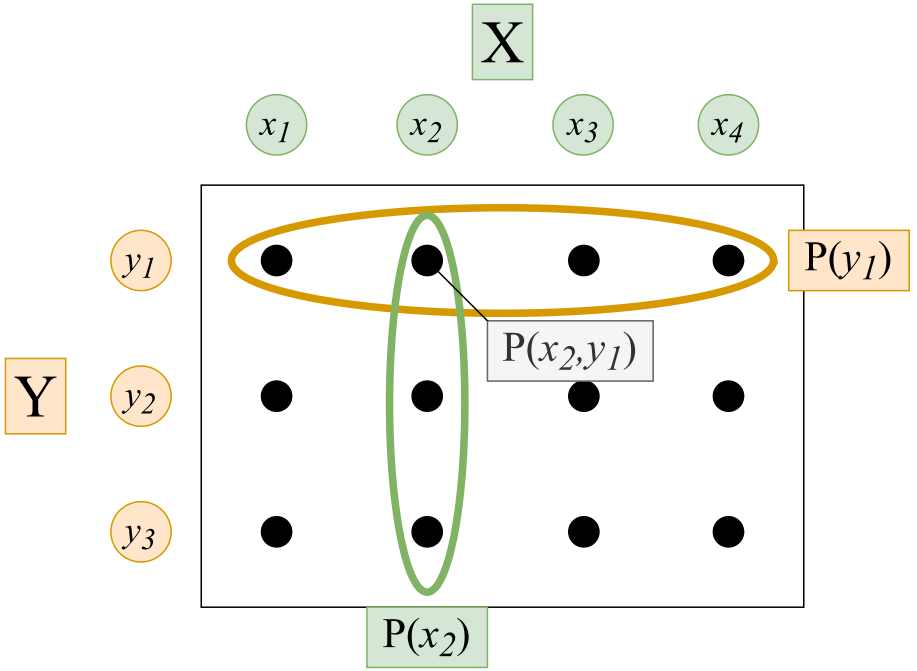
Joint, Conditional, Marginal Probability

For two random variables, a **joint probability distribution** is a distribution of all possible pairs of outputs (and analogously for more than two):

$$P(\mathrm{x} = x_2, \mathrm{y} = y_1).$$

Marginal distribution is a distribution of one (or a subset) of the random variables and can be obtained by summing over the other variable(s):

$$P(\mathrm{x} = x_2) = \sum_y P(\mathrm{x} = x_2, \mathrm{y} = y).$$



Conditional distribution is a distribution of one (or a subset) of the random variables, given that another event has already occurred:

$$P(\mathrm{x} = x_2 \mid \mathrm{y} = y_1) = P(\mathrm{x} = x_2, \mathrm{y} = y_1)/P(\mathrm{y} = y_1).$$

If $P(\mathrm{x}{=}x, \mathrm{y}{=}y) = P(\mathrm{x}{=}x) \cdot P(\mathrm{y}{=}y)$ for all x, y, random variables x, y are **independent**.

Expectation

The expectation of a function $f(x)$ with respect to a discrete probability distribution $P(\mathrm{x})$ is defined as:

$$\mathbb{E}_{\mathrm{x}\sim P}[f(x)] \stackrel{\text{def}}{=} \sum_x P(x)f(x).$$

For continuous variables, the expectation is computed as:

$$\mathbb{E}_{\mathrm{x}\sim p}[f(x)] \stackrel{\text{def}}{=} \int_x p(x)f(x)\,\mathrm{d}x.$$

If the random variable is obvious from context, we can write only $\mathbb{E}_P[x]$, $\mathbb{E}_{\mathrm{x}}[x]$, or even $\mathbb{E}[x]$.

Expectation is linear, i.e., for constants $\alpha, \beta \in \mathbb{R}$:

$$\mathbb{E}_{\mathrm{x}}[\alpha f(x) + \beta g(x)] = \alpha\mathbb{E}_{\mathrm{x}}[f(x)] + \beta\mathbb{E}_{\mathrm{x}}[g(x)].$$

Variance

Variance measures how much the values of a random variable differ from its mean $\mu = \mathbb{E}[x]$.

$$\mathrm{Var}(x) \stackrel{\text{def}}{=} \mathbb{E}\left[\left(x - \mathbb{E}[x]\right)^2\right], \text{ or more generally,}$$
$$\mathrm{Var}(f(x)) \stackrel{\text{def}}{=} \mathbb{E}\left[\left(f(x) - \mathbb{E}[f(x)]\right)^2\right].$$

It is easy to see that

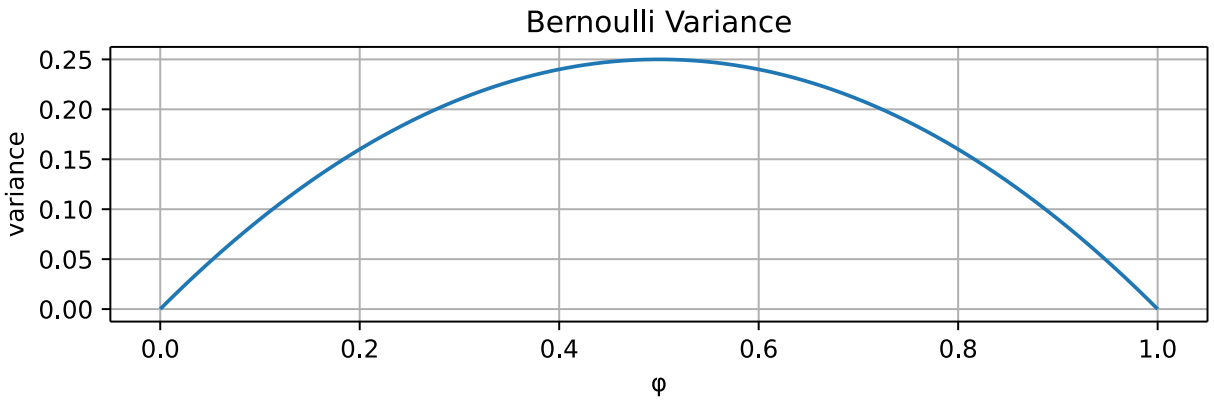
$$\begin{aligned}
&\mathrm{Var}(x) = \mathbb{E}\left[x^2\right] - \left(\mathbb{E}[x]\right)^2, \\
&\mathrm{Var}(\alpha x) = \alpha^2 \cdot \mathrm{Var}(x) && \text{for any constant} \quad \alpha \in \mathbb{R}, \\
&\mathrm{Var}(x + \beta) = \mathrm{Var}(x) && \text{for any constant} \quad \beta \in \mathbb{R}.
\end{aligned}$$

Variance is connected to $\mathbb{E}[x^2]$, the **second moment** of a random variable – it is in fact a **centered** second moment.

Bernoulli Distribution

The Bernoulli distribution is a distribution over a binary random variable. It has a single parameter $\varphi \in [0, 1]$, which specifies the probability that the random variable is equal to 1.

$$P(x) = \varphi^x (1 - \varphi)^{1-x}$$
$$\mathbb{E}[x] = \varphi$$
$$\mathrm{Var}(x) = \varphi(1 - \varphi)$$



Categorical Distribution

Extension of the Bernoulli distribution to random variables taking one of K different discrete outcomes. It is parametrized by $\boldsymbol{p} \in [0,1]^K$ such that $\sum_{i=0}^{K-1} p_i = 1$.

We represent outcomes as vectors $\in \{0,1\}^K$ in the **one-hot encoding**. Therefore, an outcome $x \in \{0, 1, \ldots, K-1\}$ is represented as a vector

$$\mathbf{1}_x \stackrel{\text{def}}{=} \big([i = x]\big)_{i=0}^{K-1} = \big(\underbrace{0, \ldots, 0}_{x}, 1, \underbrace{0, \ldots, 0}_{K-x-1}\big).$$

The outcome probability, mean, and variance are very similar to the Bernoulli distribution.

$$P(\boldsymbol{x}) = \prod_{i=0}^{K-1} p_i^{x_i}$$
$$\mathbb{E}[x_i] = p_i$$
$$\mathrm{Var}(x_i) = p_i(1 - p_i)$$

Normal (or Gaussian) Distribution

Distribution over real numbers, parametrized by a mean μ and variance σ^2:

$$\mathcal{N}(x;\mu,\sigma^2) = \sqrt{\frac{1}{2\pi\sigma^2}}\exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

For standard values $\mu = 0$ and $\sigma^2 = 1$ we get $\mathcal{N}(x;0,1) = \sqrt{\frac{1}{2\pi}}e^{-\frac{x^2}{2}}$.

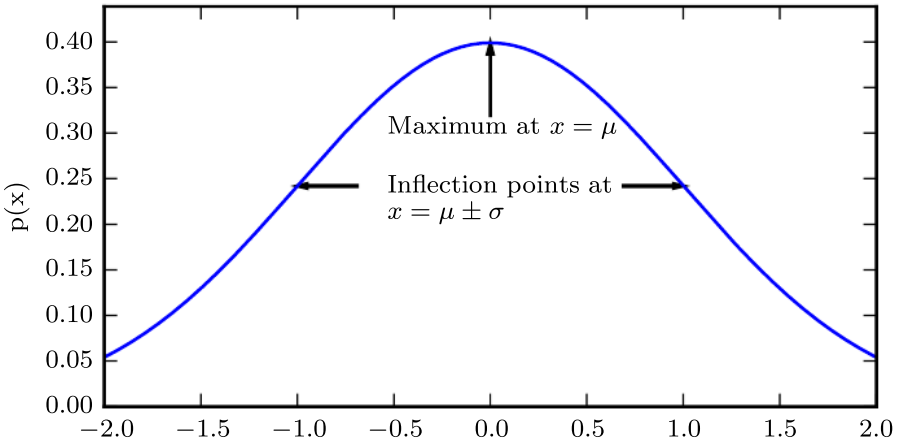


Figure 3.1 of "Deep Learning" book, https://www.deeplearningbook.org

Why Normal Distribution

Central Limit Theorem

The sum of independent identically distributed random variables with finite variance converges to normal distribution.

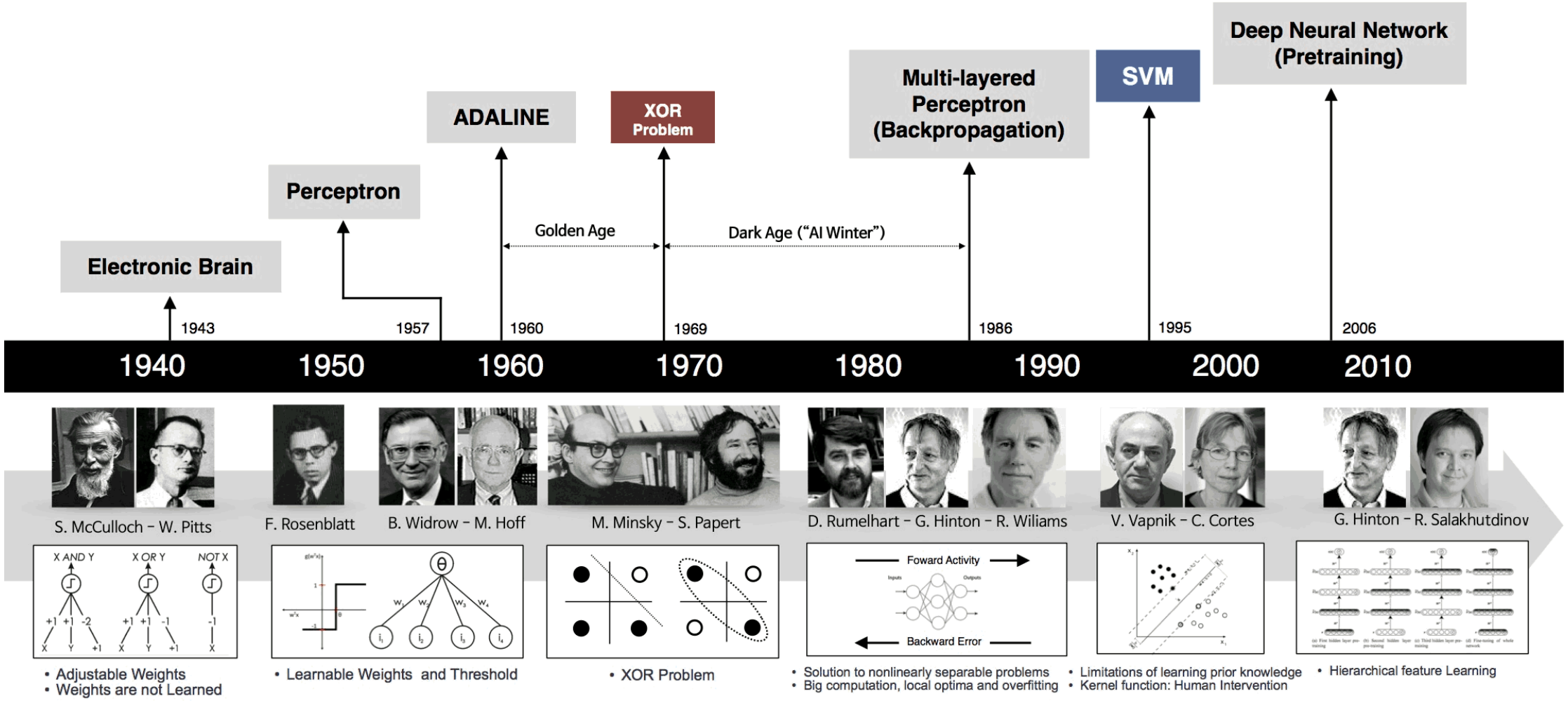
Principle of Maximum Entropy

Given a set of constraints, a distribution with maximal entropy fulfilling the constraints can be considered the most general one, containing as little additional assumptions as possible.

Considering distributions on all real numbers with a given mean and variance, it can be proven (using variational inference) that such a distribution with **maximum entropy** is exactly the normal distribution.

Neural Network Architecture à la '80s

Introduction to Deep Learning History



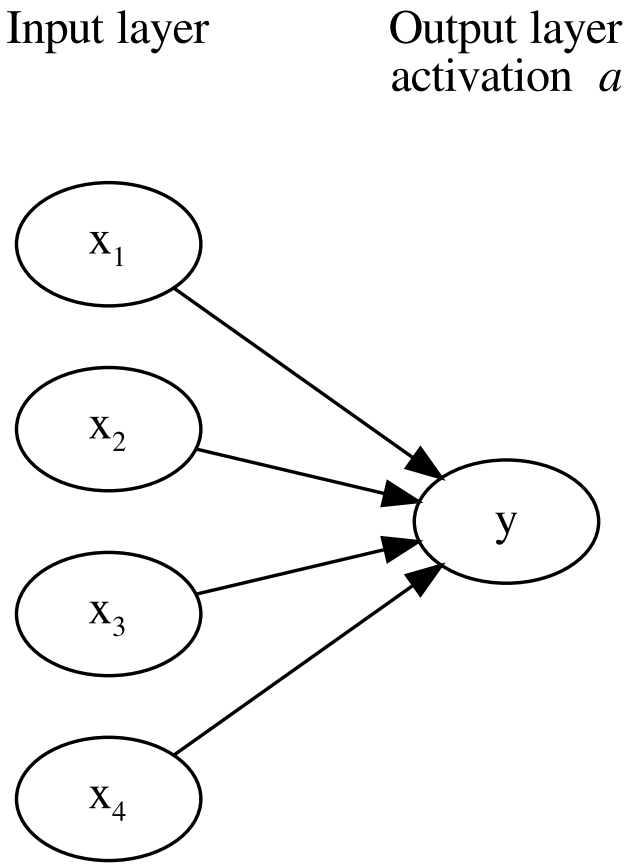
Modified from https://www.slideshare.net/deview/251-implementing-deep-learning-using-cu-dnn/4

Perceptron – Extra Simple Neural Network

- Assume we have an input node for every input feature.
- Additionally, we have an output node for every model output.
- Every input node and output node are connected with a directed edge, and every edge has an associated weight.
- Value of every (output) node is computed by summing the values of predecessors multiplied by the corresponding weights, added to a bias of this node, and finally passed through an activation function a:

$$y = a\left(\sum_j x_j w_j + b\right)$$

or in vector form $y = a(\boldsymbol{x}^T \boldsymbol{w} + b)$, or for a batch of examples $\boldsymbol{X}$, $\boldsymbol{y} = a(\boldsymbol{X}\boldsymbol{w} + b)$.

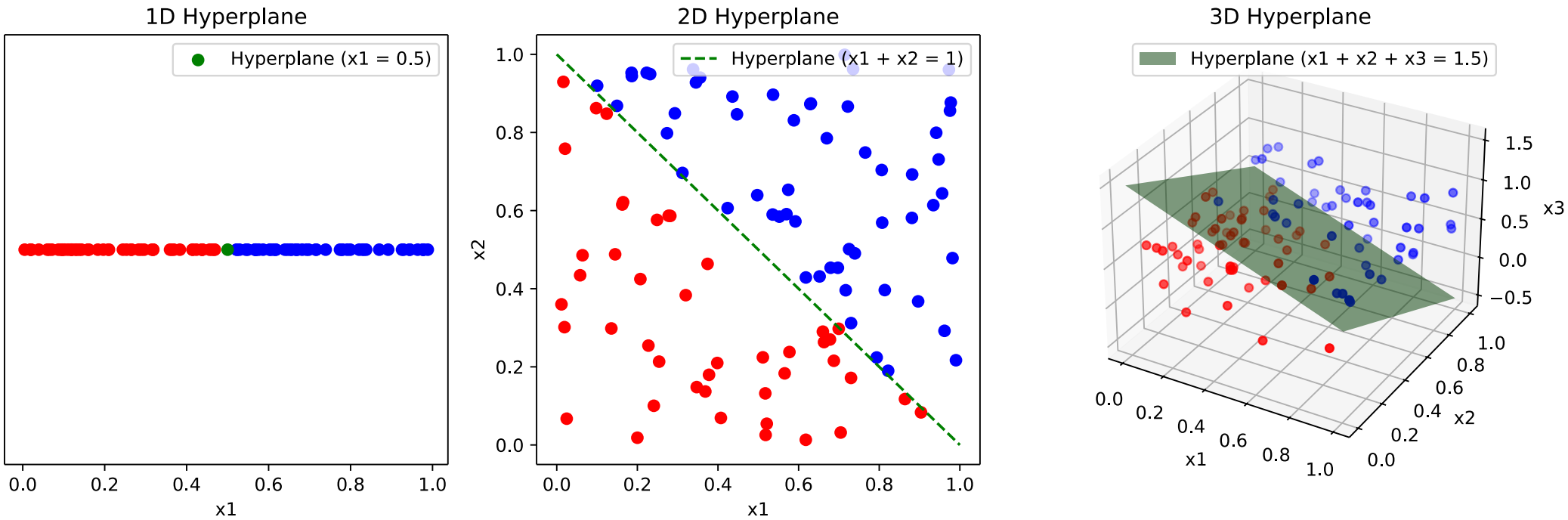


Perceptron – Separating Hyperplane

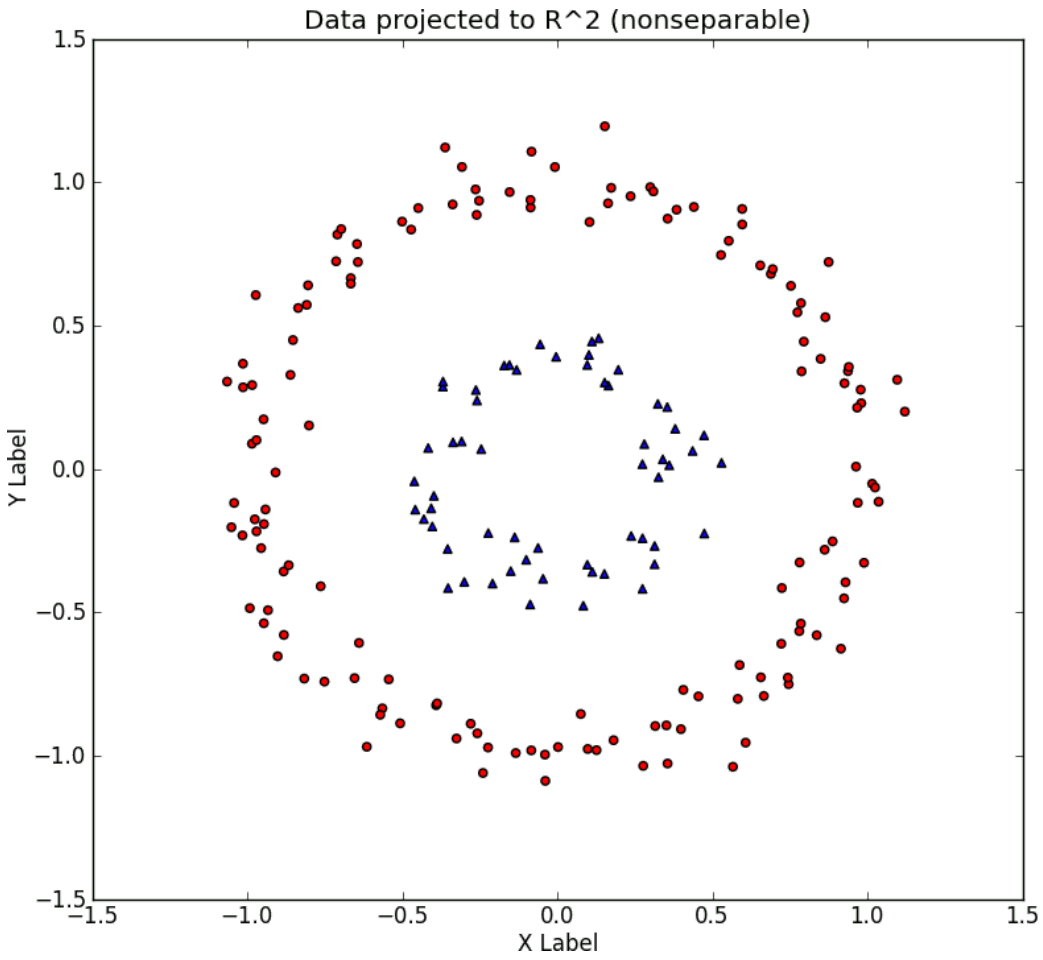
To classify an example in two classes, we compare the output $y = a(\boldsymbol{x}^T\boldsymbol{w} + b)$ with some threshold z, and choose the resulting class depending on whether $y > z$ or $y < z$.

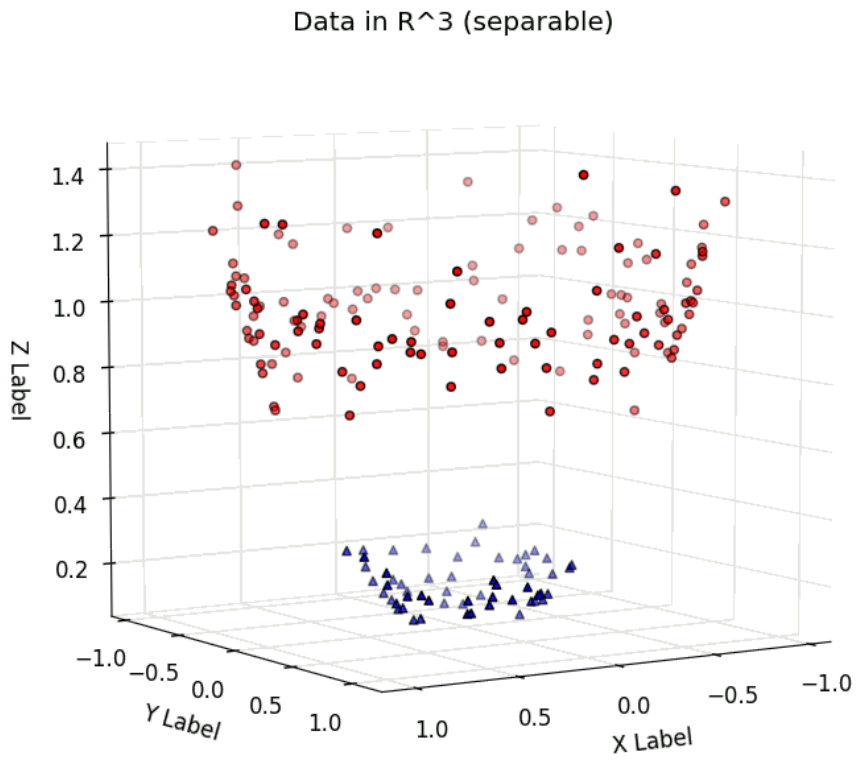
When the activation function is monotonous, this is equal to comparing $\boldsymbol{x}^T\boldsymbol{w} + b > a^{-1}(z)$.

The resulting classes are therefore divided by a *separating hyperplane* $\boldsymbol{x}^T\boldsymbol{w} + c$.



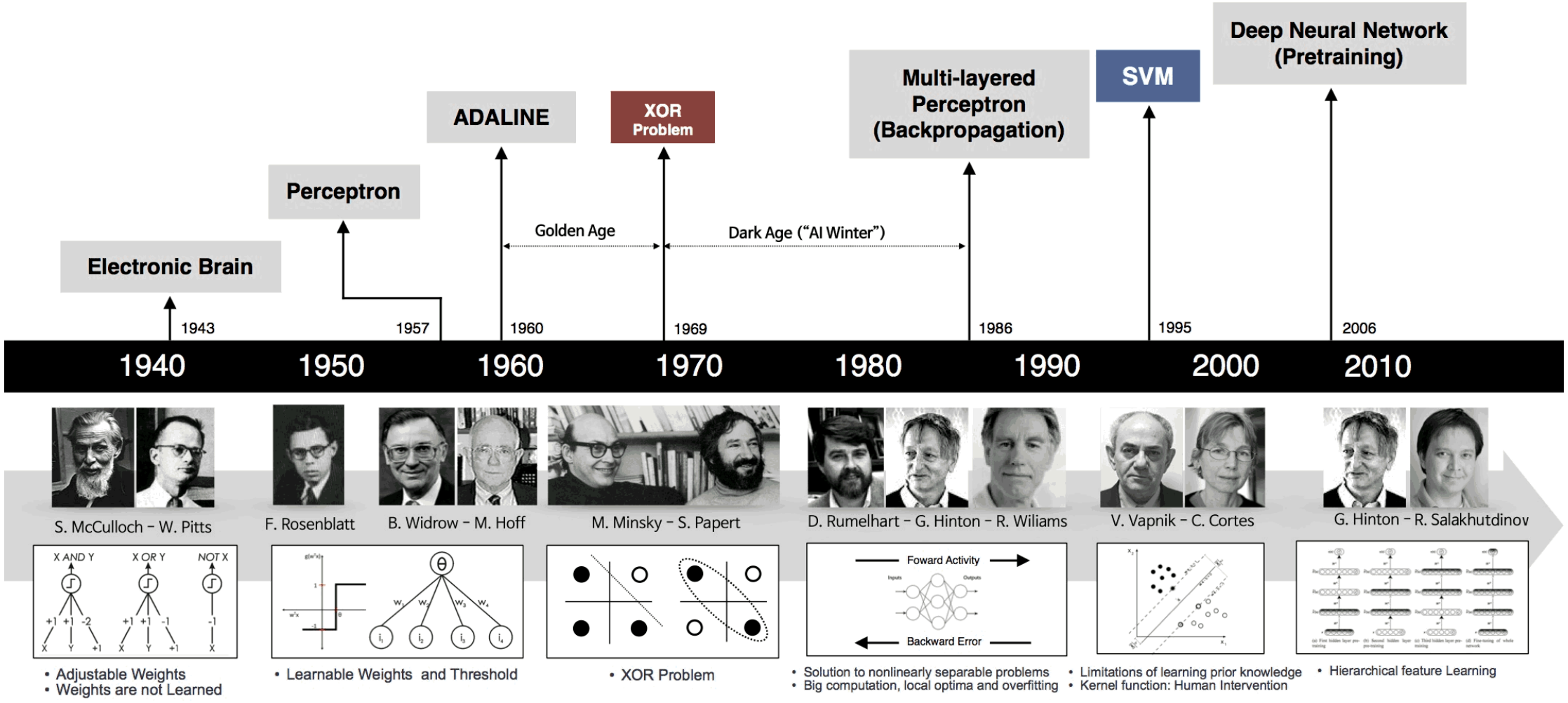
Perceptron – Linearly Separable and Nonseparable Data





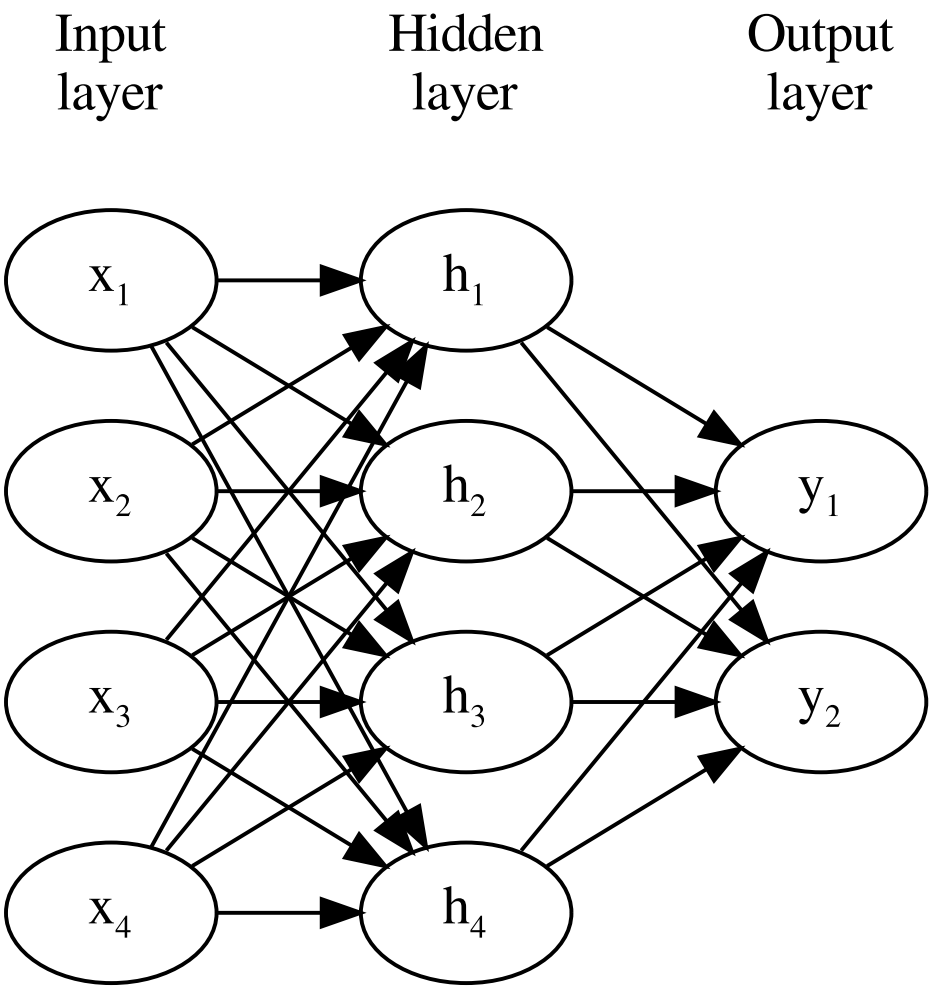
*https://miro.medium.com/v2/1*JVZ4FXVRlr1oN-4ffq_kNQ.png*

Introduction to Deep Learning History



Modified from https://www.slideshare.net/deview/251-implementing-deep-learning-using-cu-dnn/4

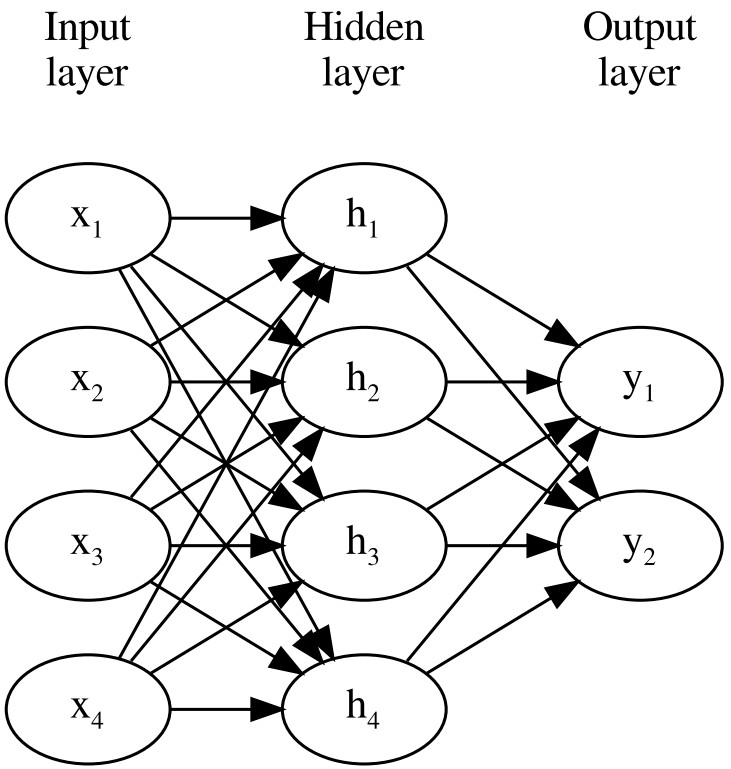
Neural Network Architecture à la '80s



Neural Network Architecture

The computation is performed analogously to the perceptron

$$h_i = f\left(\sum_j x_j w_{j,i}^{(h)} + b_i^{(h)}\right),$$
$$y_i = a\left(\sum_j h_j w_{j,i}^{(y)} + b_i^{(y)}\right),$$



or in matrix form

$$\boldsymbol{h} = f\left(\boldsymbol{x}^T \boldsymbol{W}^{(h)} + \boldsymbol{b}^{(h)}\right),$$
$$\boldsymbol{y} = a\left(\boldsymbol{h}^T \boldsymbol{W}^{(y)} + \boldsymbol{b}^{(y)}\right),$$

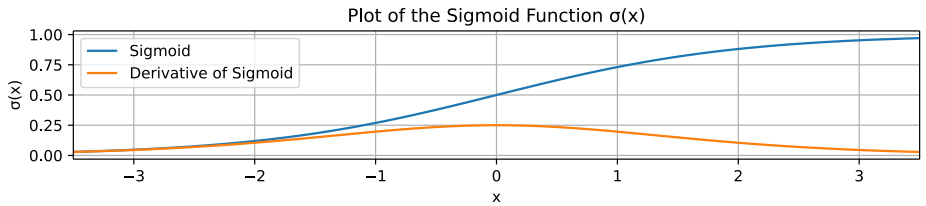
or for a whole batch of inputs $\boldsymbol{H} = f\left(\boldsymbol{X}\boldsymbol{W}^{(h)} + \boldsymbol{b}^{(h)}\right)$ and $\boldsymbol{Y} = a\left(\boldsymbol{H}\boldsymbol{W}^{(y)} + \boldsymbol{b}^{(y)}\right)$.

The $\boldsymbol{W}^{(h)} \in \mathbb{R}^{|input|\cdot|hidden|}$ is a matrix of weights and $\boldsymbol{b}^{(h)} \in \mathbb{R}^{|hidden|}$ a vector of biases of the first layer; $\boldsymbol{W}^{(y)} \in \mathbb{R}^{|hidden|\cdot|output|}$ and $\boldsymbol{b}^{(y)} \in \mathbb{R}^{|output|}$ are parameters of the second layer.

Output Layers

- none (linear regression if there are no hidden layers);
- σ (sigmoid; logistic regression if there are no hidden layers)

$$\sigma(x) \stackrel{\text{def}}{=} \frac{1}{1+e^{-x}}$$



is used to model a Bernoulli distribution, i.e., the probability φ of one of the two outcomes;

- the input of the sigmoid is called a **logit**, and it has a value of $\log \frac{\varphi}{1-\varphi}$

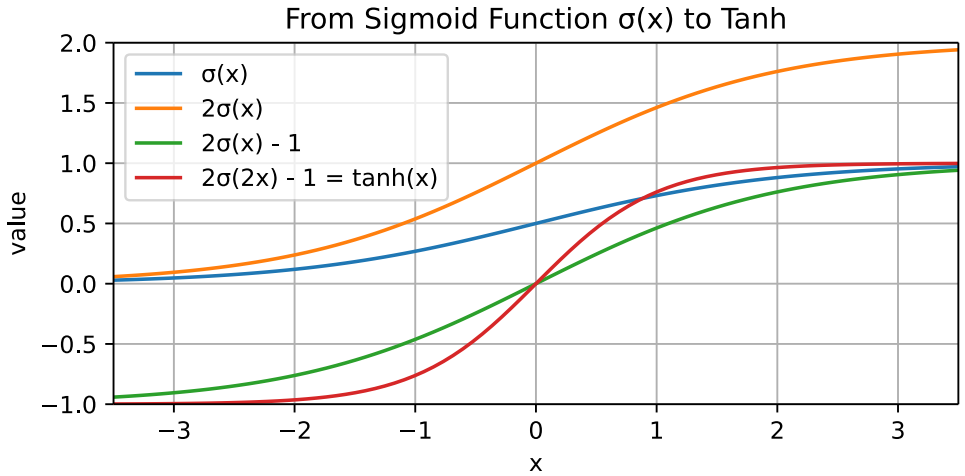
- $\mathrm{softmax}$ (maximum entropy model if there are no hidden layers)

$$\mathrm{softmax}(\boldsymbol{x}) \propto e^{\boldsymbol{x}}, \qquad \mathrm{softmax}(\boldsymbol{x})_i \stackrel{\text{def}}{=} \frac{e^{x_i}}{\sum_j e^{x_j}}$$

is used to model probability distribution $\boldsymbol{p}$; its input is called a **logit**, $\log(\boldsymbol{p}) + c$.

Hidden Layers

- none: does not help, composition of linear/affine mapping is a linear/affine mapping
- σ: does not work great – nonsymmetrical, repeated application converges to the fixed point $x = \sigma(x) \approx 0.659$, and $\frac{d\sigma}{dx}(0) = 1/4$
- $\tanh$
  - result of making σ symmetrical and making the derivative in zero 1
  - $\tanh(x) = 2\sigma(2x) - 1$



- ReLU: $\max(0, x)$

Universal Approximation Theorem '89

Let $\varphi(x) : \mathbb{R} \to \mathbb{R}$ be a nonconstant, bounded and nondecreasing continuous function. (Later a proof was given also for $\varphi = \mathrm{ReLU}$ and even for any nonpolynomial function.)

For any $\varepsilon > 0$ and any continuous function $f : [0,1]^D \to \mathbb{R}$, there exists $H \in \mathbb{N}$, $\boldsymbol{v} \in \mathbb{R}^H$, $\boldsymbol{b} \in \mathbb{R}^H$ and $\boldsymbol{W} \in \mathbb{R}^{D \times H}$, such that if we denote

$$F(\boldsymbol{x}) = \boldsymbol{v}^T \varphi(\boldsymbol{x}^T \boldsymbol{W} + \boldsymbol{b}) = \sum_{i=1}^{H} v_i \varphi(\boldsymbol{x}^T \boldsymbol{W}_{*,i} + b_i),$$

where φ is applied element-wise, then for all $\boldsymbol{x} \in [0,1]^D$:

$$|F(\boldsymbol{x}) - f(\boldsymbol{x})| < \varepsilon.$$

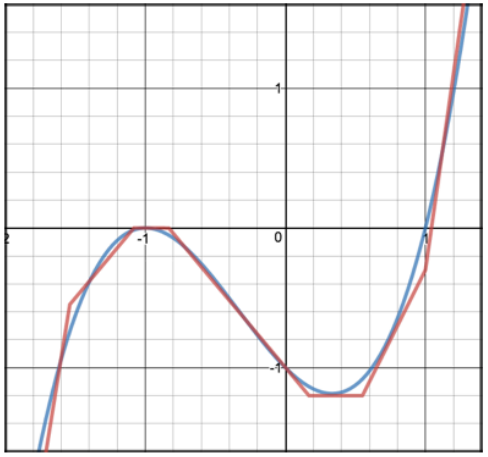
One Possible Interpretation

It is always possible to create features using just a single linear layer followed by a nonlinearity, such that the resulting dataset is always linearly separable.

Universal Approximation Theorem for ReLUs

Sketch of the proof:

- If a function is continuous on a closed interval, it can be approximated by a sequence of lines to arbitrary precision.



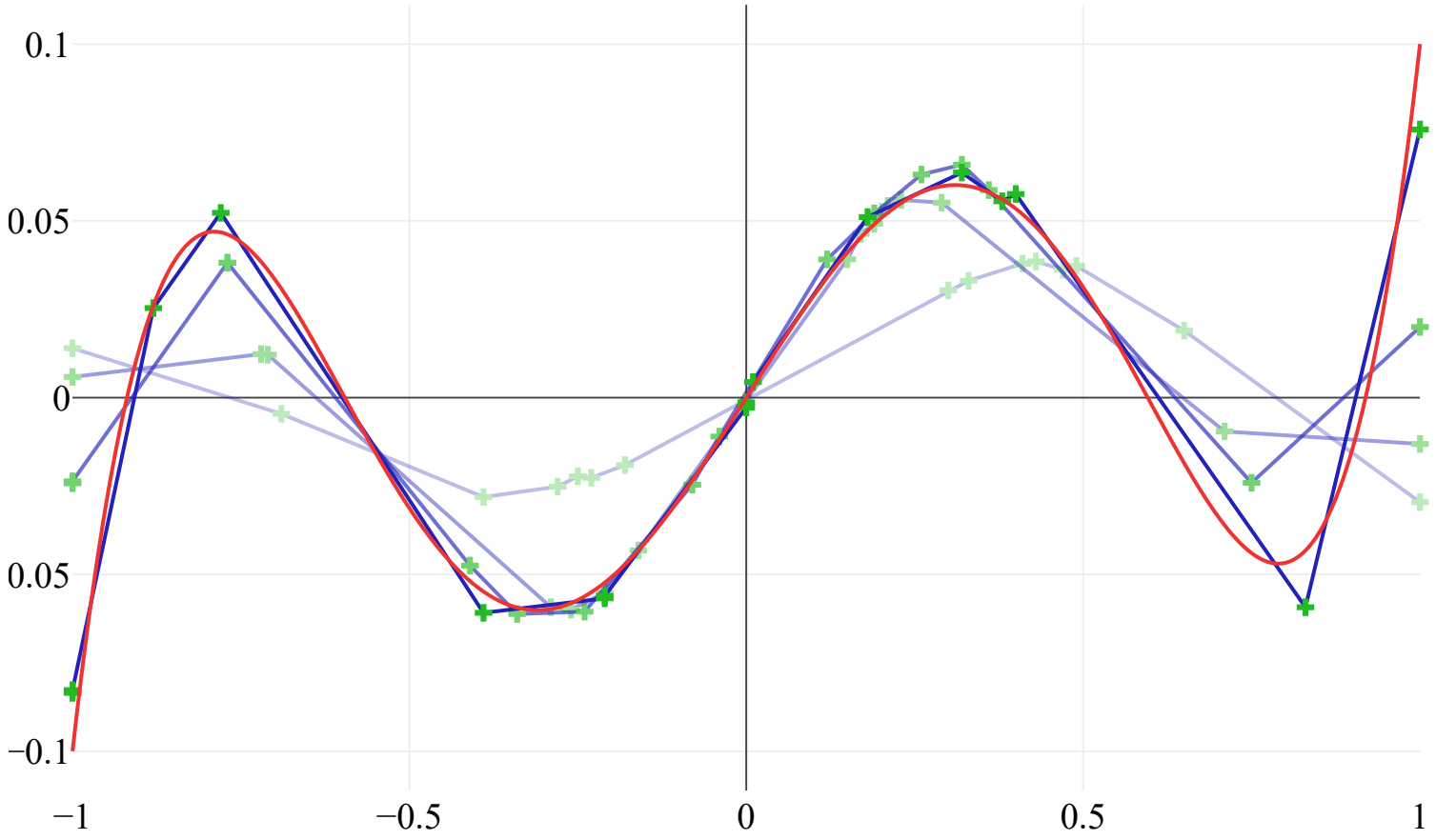
$$n_1(x) = Relu(-5x - 7.7)$$
$$n_2(x) = Relu(-1.2x - 1.3)$$
$$n_3(x) = Relu(1.2x + 1)$$
$$n_4(x) = Relu(1.2x - .2)$$
$$n_5(x) = Relu(2x - 1.1)$$
$$n_6(x) = Relu(5x - 5)$$

$$Z(x) = -n_1(x) - n_2(x) - n_3(x) + n_4(x) + n_5(x) + n_6(x)$$

*https://miro.medium.com/max/844/1*lihbPNQgl7oKjpCsmzPDKw.png*

- However, we can create a sequence of k linear segments as a sum of k ReLU units – on every endpoint a new ReLU starts (i.e., the input ReLU value is zero at the endpoint), with a tangent which is the difference between the target tangent and the tangent of the approximation until this point.

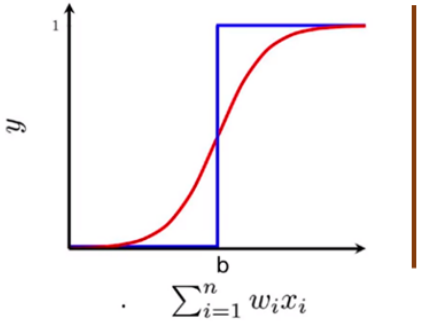
Evolving ReLU Approximation



Universal Approximation Theorem for Squashes

Sketch of the proof for a squashing function $\varphi(x)$ (i.e., nonconstant, bounded and nondecreasing continuous function like sigmoid):

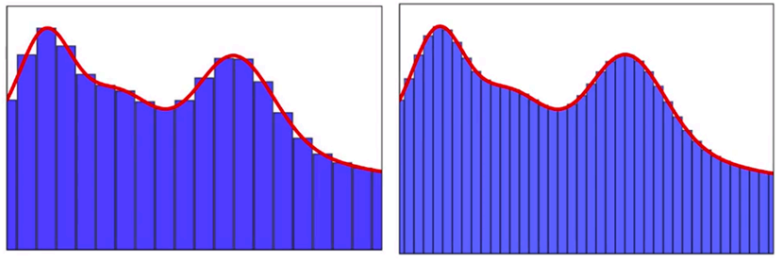
- We can prove φ can be arbitrarily close to a hard threshold by compressing it horizontally.



$$y = \frac{1}{1+e^{-(w^T x+b)}}$$

https://hackernoon.com/hn-images/1*N7dfPwbiXC-Kk4TCbfRerA.png

- Then we approximate the original function using a series of straight line segments



https://hackernoon.com/hn-images/1*hVuJgUTLUFWTMmJhl_fomg.png

Main Takeaways

Neural networks are just a model for describing computation of outputs from given inputs:

- strong enough to approximate any reasonable function,
- reasonably compact,
- allows heavy parallelization during execution (GPUs, TPUs, ...).

Nearly all the time, neural networks generate a *probability distribution* on output:

- output distribution determines the output activation function,
- distributions allow small changes during training,
- during prediction, we usually take the most probable outcome (class/label/...).

When there is enough data, neural networks are currently the best performing machine learning model, especially when the data are high-dimensional (images, speech, text).

- For tabular data, you should also consider gradient-boosted decision trees.